ingenieur
wissenschaften
htw saar

Hochschule für
Technik und Wirtschaft
des Saarlandes
University of
Applied Sciences

Bachelor-Thesis

zur Erlangung des akademischen Grades
Bachelor of Science (B. Sc.)
an der Hochschule für Technik und Wirtschaft des Saarlandes
im Studiengang Praktische Informatik
der Fakultät für Ingenieurwissenschaften

Evaluierung lokaler KI-Modelle auf Android für BCI-Anwendungen

vorgelegt von
Ahmed Chaabo

betreut und begutachtet von
Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J.Strauss
Prof. Dr.-ing. Klaus Berberich

Saarbrücken, Tag. Monat Jahr

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.
Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.
Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, Tag. Monat Jahr

Ahmed Chaabo

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: `http://plg.uwaterloo.ca/~migod/research/beckOOPSLA.html`.

We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.

— Donald E. Knuth

Danksagung

Zuallererst möchte ich mich bei meinen Betreuern, Tobias Grün und Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J. Strauss, sowie dem Forschungsinstitut SNN-Unit für die Möglichkeit bedanken, diese Arbeit im Rahmen des Bachelorstudiengangs Praktische Informatik an der Hochschule für Technik und Wirtschaft des Saarlandes zu schreiben. Ein weiterer Dank geht an meine Kollegen. Ohne deren Unterstützung wäre ich vielleicht nicht hier.

Ein besonderer Dank geht an meine Familie, da sie die Grundlage dafür geschaffen hat, dass ich heute dort bin, wo ich bin. Als ich jung war, waren meine Eltern die Einzigen, die volles Vertrauen in mich hatten. Dafür bin ich noch heute sehr dankbar, denn es motiviert mich, noch mehr Gas zu geben.

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Motivation . . . 1
- 1.2 Zielsetzung . . . 1
- 1.3 Aufbau der Arbeit . . . 1

2 Grundlagen **3**
- 2.1 Elektroenzephalographie (EEG) . . . 3
  - 2.1.1 Eigenschaften des Signals . . . 3
  - 2.1.2 Herausforderungen durch Rauschen und Artefakte . . . 3
- 2.2 Frequenzbänder . . . 3
- 2.3 Frequenzanalyse . . . 4
  - 2.3.1 Fourier-Transformation . . . 4
  - 2.3.2 Fensterung und spektrale Leckage . . . 4
  - 2.3.3 Welch-Methode . . . 4
- 2.4 Lokale Sprachmodelle auf mobilen Geräten . . . 5
  - 2.4.1 Large Language Models . . . 5
  - 2.4.2 On-Device-Inferenz und Quantisierung . . . 5
  - 2.4.3 Modell und Laufzeitumgebung . . . 6
  - 2.4.4 Systemintegrierte und frei deploybare Modelle . . . 6
- 2.5 Brain-Computer-Interface (BCI) . . . 7

3 Stand der Technik **9**
- 3.1 On-Device-Inferenz-Lösungen . . . 9
  - 3.1.1 Systemintegrierte Lösung . . . 9
  - 3.1.2 Frei deploybare Lösung . . . 9
- 3.2 Die Gemma-Modellfamilie . . . 9
- 3.3 Ausgangslage: bestehende Anwendung . . . 10

4 Methodik **13**
- 4.1 Merkmalsextraktion . . . 13
  - 4.1.1 Segmentierung . . . 13
  - 4.1.2 Spektrale Merkmale . . . 14
  - 4.1.3 Zeitstatistische Merkmale . . . 14
  - 4.1.4 Aufbereitung für das Sprachmodell . . . 15
- 4.2 Modellauswahl . . . 15
- 4.3 Anwendungsszenarien . . . 16
- 4.4 Messprotokoll . . . 16

5 Implementierung **17**
- 5.1 Architektur . . . 17
- 5.2 Implementierung der Merkmalsextraktion . . . 18
- 5.3 Integration des Sprachmodells . . . 19
- 5.4 Benutzeroberfläche . . . 19

6 Evaluation **21**
6.1 Versuchsaufbau 21
6.2 Latenz 21
6.3 Ressourcenverbrauch 21
6.4 Stabilität 21
6.5 Ausgabequalität 21

7 Diskussion **23**
7.1 Einordnung der Ergebnisse 23
7.2 Einschränkungen 23
7.3 Handlungsempfehlungen 23

8 Fazit und Ausblick **25**
8.1 Fazit 25
8.2 Ausblick 25

Literatur **27**

Abbildungsverzeichnis **29**

Tabellenverzeichnis **29**

Listings **29**

Abkürzungsverzeichnis **31**

A Erster Abschnitt des Anhangs **35**

1 Einleitung

1.1 Motivation

1.2 Zielsetzung

1.3 Aufbau der Arbeit

2 Grundlagen

In diesem Kapitel werden die Grundlagen, die für das Verständnis der Arbeit benötigt werden, vermittelt. Im Folgenden werden zentrale Konzepte vorgestellt, auf denen die weiteren Kapitel aufbauen. Ziel dieses Kapitels ist es, ein allgemeines Verständnis aufzubauen, um eine fundierte Wissensbasis für die darauffolgenden Teile der Arbeit zu schaffen.

2.1 Elektroenzephalographie (EEG)

Die Elektroenzephalographie (EEG) ist ein Verfahren zur Aufzeichnung der elektrischen Aktivität des menschlichen Gehirns. Die aufgezeichneten Signale bestehen aus Spannungsschwankungen im Mikrovoltbereich, welche aus der synchronen elektrischen Aktivität von Neuronen resultieren [2]. Die Messung erfolgt über Elektroden, die auf der Kopfhaut platziert werden und die Potenzialdifferenzen zwischen verschiedenen Punkten erfassen [1].

2.1.1 Eigenschaften des Signals

EEG-Signale haben die Eigenschaft, extrem gerige Amplitude im Mikrovoltbereich zu besitzen, weshalb es bei der Erfassung systematisch verstärkt werden muss [1, 2]. Das Frequenzspektrum erstreckt sich dabei meist von 0,1 Hz bis 100 Hz [1].

2.1.2 Herausforderungen durch Rauschen und Artefakte

In seiner Rohform ist ein EEG-Signal stark von Rauschen geprägt, weshalb es ein unzureichendes Signal-Rausch-Verhältnis (SNR) aufweist [2]. Da die erwarteten neuronalen Aktivitäten schwach sind, werden sie schnell von amplitudenstarken physiologischen Artefakten (wie z. B. Muskel- oder Augenbewegungen) sowie technischen Störeinflüssen überlagert, was zu Verzerrungen des Originalsignals führt [1].

2.2 Frequenzbänder

Frequenzbänder sind Frequenzbereiche, in welche die Gehirnwellen nach ihrer Frequenz eingeteilt werden. Die elektrischen Schwingungen werden in Hertz (Hz) gemessen, also in Ausschlägen pro Sekunde [1]. Die gezielte Auswertung spezifischer Frequenzbänder dient somit der fundierten Beurteilung von Gehirnaktivitäten [1].

Das EEG-Spektrum wird in jeweils 5 Frequenzbänder unterteilt [1]:

Tabelle 2.1: Die fünf EEG-Frequenzbänder und ihre assoziierten Zustände [1]

Band	Frequenzbereich	Assoziierter Zustand
Delta	0,5–4 Hz	Tiefer, sehr entspannter und erholsamer Zustand
Theta	4–8 Hz	Leichte Schlafphase oder Meditation; Zustand zwischen Schlaf und Wachsein
Alpha	8–12 Hz	Ruhiger, stressfreier Zustand; in der Großhirnrinde deutlich nachweisbar, am ausgeprägtesten bei geschlossenen Augen
Beta	12–30 Hz	Aktives Denken und Problemlösen
Gamma	ab 30 Hz	Konzentration und Multitasking

2.3 Frequenzanalyse

Wichtig ist es zu bestimmen, welche Frequenzbänder in EEG-Signalen wie stark vertreten sind. Im Folgenden wird beschrieben, welche Verfahren dafür nötig sind.

2.3.1 Fourier-Transformation

Die Fast Fourier Transform (FFT) ist ein mathematisches Verfahren, das komplexe Signale in ihre sinusförmigen Bestandteile zerlegt und sie vom Zeitbereich in den Frequenzbereich überführt [1]. Dies wird verwendet um bei der Analyse der Gehirnströme, die spektralen eigenschaften von signalen zu untersuchen und Frequenspitzen (Peaks) zu berechnen. [1].

2.3.2 Fensterung und spektrale Leckage

Die Fensterung ist ein Verfahren, um Signale zu analysieren, deren Frequenzen sich im Laufe der Zeit verändern [1]. Die Auswertung geschieht in zeitlich verschiebbaren Abschnitten (den Fenstern) statt im gesamten Signal auf einmal [1]. Wenn ein Signal für die Datenverarbeitung abgeschnitten wird, entsteht eine spektrale Leckage. Dies geschieht häufig dann, wenn die Länge des erfassten Signalabschnitts kein exaktes Vielfaches der eigentlichen Signalperiode ist. Durch dieses harte Abschneiden wird die Energie des Signals im Frequenzspektrum verfälscht und auf einen breiteren Frequenzbereich verteilt, was zu Störfrequenzen führt. Um diese Leckage auszugleichen, nutzt man die Fensterfunktion, bei der man die Stärke des Signals an den Rändern des Abschnitts sanft abschwächt [15].

2.3.3 Welch-Methode

Um Frequenzspektren robuster zu schätzen und Rauschen zu reduzieren, wird die Welch-Methode verwendet [15]. Es ist ein Rechenverfahren, das in drei Schritten vorgeht:

1. Das ursprüngliche Signal wird in mehrere kürzere Abschnitte unterteilt, die sich teilweise überlappen dürfen [15].
2. Auf jeden Abschnitt wird eine Fensterfunktion angewendet, um dann für jeden Abschnitt einzeln das Spektrum zu berechnen [15].
3. Ganz am Ende werden die Einzelspektren zu einem Durchschnittswert gemittelt [15].

2.4 Lokale Sprachmodelle auf mobilen Geräten

Im folgenden Abschnitt werden die relevanten Grundlagen dargestellt, die für das Ausführen von Sprachmodellen auf mobilen Geräten notwendig sind. Zunächst werden große Sprachmodelle, deren Vor- und Nachteile sowie die Quantisierung vorgestellt.

2.4.1 Large Language Models

Large Language Models (LLMs) haben die Verarbeitung natürlicher Sprache (Natural Language Processing) grundlegend transformiert. LLMs sind hochkomplex und basieren auf der Transformer-Architektur [17]. Das Besondere daran ist der Self-Attention-Mechanismus (Selbstaufmerksamkeit). Dieser dient dazu, globale Abhängigkeiten zwischen Ein- und Ausgabesequenzen effizient zu modellieren [16]. Die hohe Leistungsfähigkeit der Modelle wird durch die gelernten Werte von Milliarden von Parametern erreicht. Ihnen werden herausragende Eigenschaften im Sprachverstehen, in der Textsynthese, im logischen Schlussfolgern sowie in der generativen Texterstellung verliehen [17]. Dadurch entstehen Herausforderungen, die es zu bewältigen gilt, etwa wenn man sie auf ressourcenbeschränkten Endgeräten wie Smartphones (sogenannten Edge-Geräten) zum Laufen bringen möchte. So benötigen LLMs enorm viel Speicherplatz und sehr viel Rechenleistung [17].

In der maschinellen Sprachverarbeitung werden Texte in Token zerlegt, die als kleinste Informationseinheiten dienen. So wird im Fachartikel [12] gesagt: „Ein Token kann für ein einzelnes Zeichen, einen Teil eines Wortes (z.,B. ein Suffix oder Präfix), ein ganzes Wort oder sogar eine kurze, aus mehreren Wörtern bestehende Phrase stehen.“ Damit die benötigte Rechenleistung reduziert wird, enthält jedes Token eine eindeutige numerische Identifikationsnummer (ID), anhand derer das Modell die Sprache lernt und verarbeitet [12].

Das Kontextfenster ist die maximale Anzahl an Token, die das Modell gleichzeitig verarbeiten kann [12]. Es ist sozusagen das Arbeitsgedächtnis des Modells. Sollte das Kontextfenster überschritten werden, werden die älteren Informationen gekürzt oder zusammengefasst, damit das Modell weiterhin im selben Kontext arbeiten kann [12].

2.4.2 On-Device-Inferenz und Quantisierung

On-Device-Inferenz beschreibt die lokale ausführung von KI-Modellen direkt auf den lokalen Endgeräten (beispielsweise einem Smartphone), anstatt die Daten zur Verarbeitung an einen externen Cloud-Server zu senden [17].

Dieser Ansatz bietet daher drei wesentliche Vorteile:

- **Datenschutz:** Da die Daten das Gerät nicht verlassen, wird die Privatsphäre der Nutzer besser geschützt [17].
- **Offline-Fähigkeit:** Auch ohne Internetverbindung ist die Anwendung nutzbar [17].
- **Kostenersparnis:** Es fallen keine Kosten durch die Abhängigkeit von Cloud-Diensten an [17].

Neben den Vorteilen gibt es auch gewisse Herausforderungen, die man zu bewältigen hat:

- **Speicher und Rechenleistung:** Da LLMs sehr große Sprachmodelle sind, benötigen sie aufgrund ihrer hohen Anzahl an Parametern ausreichend großen Arbeitsspeicher (RAM) und Rechenkapazität. Oft ist dies mehr, als herkömmliche Smartphones bieten [17].
- **Vielfalt der Endgeräte:** Verschiedene Endgeräte besitzen unterschiedliche Prozessoren und Spezialchips. Eine Lösung zu programmieren, die auf allen Endgeräten funktioniert, ist sehr schwierig [17].
- **Katastrophales Vergessen:** Ein häufiges und bekanntes Phänomen ist, dass ein Modell unweigerlich zuvor erlerntes Wissen vergisst, wenn es neue Informationen lernt [17].

Damit diese Modelle auf Mobilgeräten laufen können, ist die sogenannte Quantisierung ein essenzielles Verfahren. Die Dateigröße des Modells wird reduziert, indem die Genauigkeit der gespeicherten Zahlenwerte künstlich verringert wird. Wechselt man beispielsweise von einer 32-Bit- auf eine 4-Bit-Speicherung, sinkt der Arbeitsspeicherbedarf und es kommt zu schnelleren Berechnungen auf dem Endgerät [17].

2.4.3 Modell und Laufzeitumgebung

Der Unterschied zwischen einem Modell und einer Runtime (Laufzeitumgebung) lässt sich grundlegend wie folgt beschreiben:

- **Modell:** Ein Modell ist die Basis, die durch das Training Fähigkeiten wie Textverständnis, logisches Schließen oder die Generierung von Inhalten erlernt hat [17].
- **Runtime:** Die Runtime ist hingegen die Software, die das Modell auf dem Endgerät ausführt [9, 17]. Sie stellt die notwendigen Ressourcen und Schnittstellen bereit, um das Modell zu betreiben [17].

2.4.4 Systemintegrierte und frei deploybare Modelle

Grundsätzlich gibt es zwei Kategorien lokaler Sprachmodelle die unterschieden werken können:

- **Systemintegrierte Modelle:** zum einen gibt es systemintegrierte Modelle, die tief in das Betriebssystem integriert sind. Ein Beispiel hierfür ist das Modell Gemini Nano, welches über den Android-Systemdienst AICore ausgeführt und lokal verwaltet wird [8]. Der architektonische Vorteil dieses Ansatzes besteht darin, dass Entwickler keine eigene Runtime bereitstellen müssen, sondern über standardisierte Programmierschnittstellen (APIs) auf das Modell zugreifen können [8]. Das Betriebssystem übernimmt dabei die zentrale Verwaltung sowie Aktualisierung, was den Speicher- und Ressourcenbedarf der einzelnen Applikation erheblich reduziert.
- **Frei deploybare Modelle:** zum anderen gibt es frei deploybare Modelle, die gemeinsam mit einer passenden Runtime als Paket von Entwicklern direkt auf das Endgerät installiert und in der Applikation ausgeführt werden müssen [11]. Dadurch kann eine maximale Flexibilität gewährleistet werden, da sie unabhängiger von spezifischen Systemvorgaben oder vorinstallierten Systemdiensten funktionieren. Für die technische Umsetzung werden sogenannte „Task Bundles“ (Dateiformate wie .task oder .litertlm) genutzt [9]. Diese Datenpakete bündeln das eigentliche Modell

mit den notwendigen Tokenizer-Parametern (Werkzeuge zur Textzerlegung), um eine reibungslose Ende-zu-Ende-Verarbeitung zu gewährleisten [9]. Dieser Ansatz erhöht jedoch den Speicherbedarf der Anwendung.

2.5 Brain-Computer-Interface (BCI)

Ein Brain-Computer-Interface (BCI), zu Deutsch Gehirn-Computer-Schnittstelle, beschreibt eine Technologie, die eine Kommunikation zwischen Gehirn und Computer (externem Gerät) ermöglicht [2]. Häufig gemessen durch EEG, werden mentale Zustände und Absichten in Echtzeit erkannt. Dadurch sind Steuerungsbefehle möglich, die ganz ohne Muskelkraft auskommen [2].
Ein funktionsfähiges BCI-System erfasst die Gehirndaten, verarbeitet und analysiert diese Signale sofort in einem geschlossenen System. Der Nutzer erhält basierend auf der Klassifizierung dieser Signale eine unmittelbare Rückmeldung [2].

Vor allem körperlich stark beeinträchtigte Menschen profitieren von BCI-Systemen. Zu den bekannten Beispielen zählen [1, 2]:

- **Kommunikationssysteme:** Spezielle Systeme ermöglichen gelähmten Personen die Kommunikation mit ihrer Umwelt.
- **Bewegung und Steuerung:** BCI-Systeme erlauben die gedankliche Steuerung von Prothesen, Rollstühlen sowie das Steuern eines Cursors auf dem Computerbildschirm.
- **Assistenzsysteme:** Auch bei der Fehlerüberwachung können anhand der Gehirnaktivität die Absichten einer Notbremsung erkannt und ausgeführt werden, noch bevor das Bremspedal physisch betätigt wird.

Diese Arbeit setzt auf einem solchen EEG-basierten BCI auf und ergänzt es um ein lokales Sprachmodell, das die Messwerte in verständlichen Text überführt. Im folgenden Kapitel werden Modelle und Software-Frameworks vorgestellt, die für den mobilen Einsatz optimiert sind.

3 Stand der Technik

Dieses Kapitel gibt einen Überblick über den aktuellen Stand der Technik. Es werden zunächst die Grundlagen der On-Device-Inferenz-Lösungen erläutert, anschließend die Gemma-Modellfamilie und zum Abschluss die Ausgangslage der bestehenden Anwendung.

3.1 On-Device-Inferenz-Lösungen

Um ein Sprachmodell lokal auf einem Mobilgerät auszuführen, wird eine Laufzeitumgebung (Runtime) benötigt. Dafür stehen unterschiedliche Lösungen zur Verfügung. Diese lassen sich, wie in Abschnitt 2.4.4 dargestellt, in systemintegrierte und frei deploybare Ansätze einteilen. Beide werden im Folgenden betrachtet.

3.1.1 Systemintegrierte Lösung

Das systemintegrierte Modell Gemini Nano ist stark an die Hardware gebunden, da AICore spezifische On-Device-Hardwarekomponenten nutzt, um die Inferenz zu beschleunigen [8]. Aus diesem Grund setzt Gemini Nano bestimmte von Google freigegebene Hardware voraus. Das für diese Arbeit verwendete Testgerät Samsung Galaxy S23 erfüllt diese Anforderungen nicht, sodass Gemini Nano nicht genutzt werden kann [10].

Die Möglichkeit einer virtuellen Testumgebung wie eines Android-Emulators ist ausgeschlossen, da die physische Präsenz der Gerätehardware benötigt wird [8, 17].

Aufgrund dieser Einschränkungen steht die systemintegrierte Lösung Gemini Nano für diese diese Arbeit nicht zur Verfügung. Aus diesem Grund wird im Folgenden die frei deploybare Lösung betrachtet, die auf dem Testgerät Samsung Galaxy S23 genutzt werden kann.

3.1.2 Frei deploybare Lösung

Die MediaPipe LLM Inference API ist eine geeignete frei deploybare Lösung [9].

Abhängig vom Modelltyp und den Einstellungen können die Berechnungen entweder auf dem Hauptprozessor (CPU) oder dem Grafikprozessor (GPU) des mobilen Geräts durchgeführt werden [9].

Die Schnittstelle unterstützt von Haus aus kompakte, speziell für mobile Geräte entwickelte Modelle. Dazu gehört insbesondere die Gemma-Familie von Google, etwa Gemma 3 1B und Gemma 2 2B [9]. Für die folgenden Abschnitte sind vor allem diese beiden sehr kompakten Modelle wichtig.

3.2 Die Gemma-Modellfamilie

Die Gemma-Modellfamilie gehört zu den frei deploybaren und ressourcenschonenden Modellen, die vom Gemma-Team bei Google DeepMind entwickelt wurden [7].

Besondere Merkmale der Gemma-Modelle sind:

- **Offenheit und Effizienz:** Neben der freien Zugänglichkeit der Modelle zeichnen sie sich durch eine ressourcenschonende Ausführung auf herkömmlicher Standardhardware wie Smartphones aus [6, 7].
- **Skalierbarkeit:** Um unterschiedlichen technischen Anforderungen gerecht zu werden, sind die Modelle in mehreren Größen verfügbar. Diese werden in Milliarden von Parametern gemessen. So bietet die zweite Generation (Gemma 2) Varianten mit 2, 9 und 27 Milliarden Parametern [6]. Die dritte Generation (Gemma 3) erweitert diese Skalierung auf Modelle mit 1, 4, 12 und 27 Milliarden Parametern [7].
- **Erweiterter Funktionsumfang:** Modelle vor der dritten Generation (Gemma 1–2) unterstützten nur reine Textverarbeitung. Seit der dritten Generation sind auch multimodale Eingaben möglich, sodass neben Text auch Bilder verarbeitet werden können [7]. Zudem wurden die Fähigkeiten zur Verarbeitung unterschiedlicher Sprachen sowie zum Erfassen deutlich längerer Textzusammenhänge verbessert [7].
- **Einsatzbereitschaft:** Neben der allgemeinen Basisversion gibt es die Modelle auch in einer speziell angepassten Form, die gezielt für das Führen von Dialogen und das präzise Befolgen von Anweisungen trainiert wurde [6, 7].

3.3 Ausgangslage: bestehende Anwendung

Im Folgenden wird die bestehende Anwendung vorgestellt, die als Grundlage für die Implementierung der neuen Funktionalität dient.

- **Unicorn Hybrid Black:** Das Unicorn Hybrid Black ist die BCI-Plattform, die in dieser Arbeit verwendet wird. Er ist eine mobile Brain-Computer-Interface-Plattform der Firma g.tec [5]. Das Aufnahmegerät erfasst die elektrische Aktivität des Gehirns über acht Elektroden, die auf der Kopfhaut angebracht werden. Die Abtastrate beträgt genau 250 Hz [4]. Besonders ist vor allem die Kompaktheit des Geräts. Durch eine tragbare, drahtlose Bauform eignet er sich für den mobilen Einsatz.
- **Datenübertragung & Anbindung:** Die Übertragung der Messdaten an das Endgerät erfolgt drahtlos über Bluetooth, wobei jedes Sample aus den jeweils folgenden 17 Werten besteht [3].

Tabelle 3.1: Aufbau eines Samples des Unicorn Hybrid Black [3]

Werte	Bezeichnung	Anzahl
1–8	EEG-Kanäle	8
9–11	Beschleunigungssensor (x, y, z)	3
12–14	Gyroskop (x, y, z)	3
15	Batteriestand	1
16	Counter (fortlaufender Zähler)	1
17	Validation Indicator (valid-Flag)	1
	Gesamt	**17**

- **Bestehende Anwendung:** Die als Grundlage für diese Arbeit genutzte Anwendung ist das Ergebnis eines Projektes aus einem vorangegangenen Praktikum. Sie bildet deren Basis, indem sie eine Verbindung mit dem Unicorn Hybrid Black aufbaut, EEG-Daten aufnimmt und speichert. Diese Daten werden zugleich als Zeitverlauf oder als Live-FFT visualisiert. Die Daten werden dauerhaft gespeichert und können zusätzlich als .db- oder .json-Datei exportiert werden. Der Datenpfad verläuft dabei vom Gerät über Bluetooth zu einem Hintergrunddienst, der die eingehenden Samples in der Datenbank ablegt.
- **Beitrag dieser Arbeit:** Die bestehende Grundlage wird um Funktionalitäten erweitert. Die grundsätzliche Erweiterung ist die Verarbeitung der EEG-Daten durch lokale Sprachmodelle. Der Beitrag umfasst in diesem Fall drei neue Kernkomponenten:
  - Schicht zur Merkmalsextraktion `features`
  - Anbindung eines oder mehrerer Sprachmodelle `llm`
  - Benutzeroberfläche zur KI-gestützten Analyse `analyse`

  Im Originalcode wurden nur minimale Änderungen vorgenommen.

4 Methodik

4.1 Merkmalsextraktion

Die Merkmalsextraktion ist ein zentraler Bestandteil der Arbeit. Ein Sprachmodell kann keine EEG-Rohdaten verarbeiten, da bei einer Abtastrate von 250 Hz und acht Kanälen bereits in kurzer Zeit eine große, unstrukturierte Zahlenmenge entsteht. Bei einer nur einminütigen Aufnahme ergeben sich beispielsweise

$$\underbrace{8}_{\text{Kanäle}} \times \underbrace{250}_{\text{Hz}} \times \underbrace{60}_{\text{Sekunden}} = 120\,000 \text{ Werte.} \tag{4.1}$$

Diese Datenmenge muss zunächst in eine kompakte Form umgewandelt werden, weil sie ansonsten den Kontext des Modells überschreitet und keine fachliche Bedeutung transportiert. Die daraus resultierenden aggregierten Merkmale sind interpretierbar, deutlich token-sparsamer und können vom Sprachmodell verarbeitet werden.

4.1.1 Segmentierung

Bei einem Mittelwert über die gesamte Aufnahme würden Informationen wie der zeitliche Verlauf der Session fehlen. Aus diesem Grund wird jede Session in mehrere gleich große Segmente aufgeteilt (siehe Abbildung 4.1). Da das Signal mit einer konstanten Abtastrate von 250 Hz aufgenommen wird, entspricht eine gleiche Anzahl an Samples automatisch einer gleichen Zeitdauer. Die Segmentierung wird daher Sample-basiert durchgeführt. Dadurch ist es leichter nachvollziehbar, wie sich die Signalcharakteristik zwischen Beginn und Ende der Aufnahme verändert. Durch dieses Verfahren wird die Analyse robuster gegenüber Artefakten, da diese nur einen Teil der Session betreffen und nicht die gesamte Aufnahme verfälschen.

Damit die Aufteilung der Session möglich ist, muss die Aufnahme eine Mindestlänge haben. Damit pro Segment mindestens ein vollständiges Analysefenster für die spektrale Berechnung zur Verfügung steht, sind bei vier Abschnitten und einer Fenstergröße von 256 Werten mindestens

$$4 \text{ Abschnitte} \times 256 \text{ Werte} = 1024 \text{ Samples} \approx 4 \text{ s} \tag{4.2}$$

erforderlich. Wenn die Aufnahme zu kurz ist, um die Mindestlänge zu erfüllen, wird die Analyse abgebrochen und eine entsprechende Meldung angezeigt.

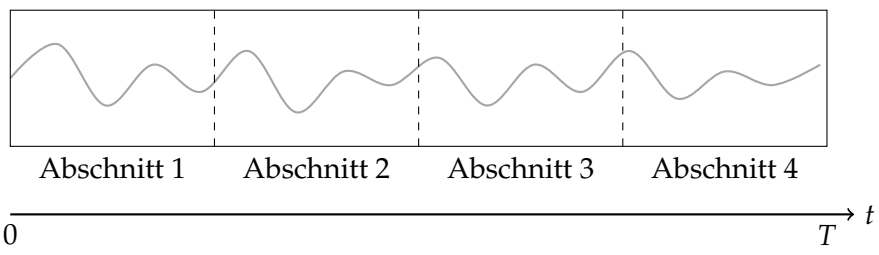


Abbildung 4.1: Aufteilung einer Session in vier gleich lange Zeitabschnitte.

4.1.2 Spektrale Merkmale

Die spektralen Merkmale werden für jeden Abschnitt bestimmt, indem die Signalenergie auf die einzelnen Frequenzbänder verteilt wird. Die kanalweise Berechnung erfolgt dabei mit der Welch-Methode [15]. Hierfür wird das Signal eines Abschnitts in überlappende Fenster unterteilt; im vorliegenden Fall beträgt die Fenstergröße 256 Samples (siehe Abbildung 4.2). Für jedes dieser Fenster wird das Leistungsspektrum berechnet, das anschließend über alle Fenster gemittelt wird. Durch diese Mittelung wird das resultierende Spektrum deutlich stabiler und weniger rauschanfällig als bei einer einzelnen FFT. Allerdings ist dann mit einer geringeren Frequenzauflösung zu rechnen [15].

Um den Gleichanteil (0 Hz) des Signals zu eliminieren, wird vor der Fourier-Transformation zunächst der Mittelwert von jedem Segment subtrahiert. Anschließend wird die Hann-Fensterfunktion (auch Hanning genannt) verwendet, um jedes Fenster zu gewichten [15]. Die Gewichtung wird verwendet, da es ansonsten zu einem direkten, abrupten Abschneiden des Signals kommt, was zu harten Übergängen an den Fenstergrenzen führt [15]. Durch diesen Vorgang entsteht eine spektrale Leckage. Die Hann-Funktion hilft gegen diese Leckage, indem sie die Amplitudenwerte an den Rändern des Fensters sanft abschwächt [15].

Die Fensterfunktion wird mathematisch wie folgt beschrieben:

$$w(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) \tag{4.3}$$

wobei (N) die Fenstergröße (256) bezeichnet [15].

Jedes Fenster wird per FFT in den Frequenzbereich überführt, um das jeweilige lokale Leistungsspektrum (Periodogramm) zu berechnen [1, 15]. Die Leistung der einzelnen Frequenz-Bins wird den fünf Frequenzbändern Delta, Theta, Alpha, Beta und Gamma zugeordnet [1]. Dies erfolgt durch das Aufsummieren der Leistung aller Frequenz-Bins innerhalb der jeweiligen Bandgrenzen. In der Regel schwanken die absoluten Leistungswerte stark und sind schwer interpretierbar. Daher wird die relative Bandleistung verwendet, die sich aus dem Verhältnis der Leistung eines Frequenzbandes zur Gesamtleistung ergibt. Dadurch lässt sich unmittelbar und robust ablesen, welches Frequenzband im untersuchten Abschnitt dominiert [1].

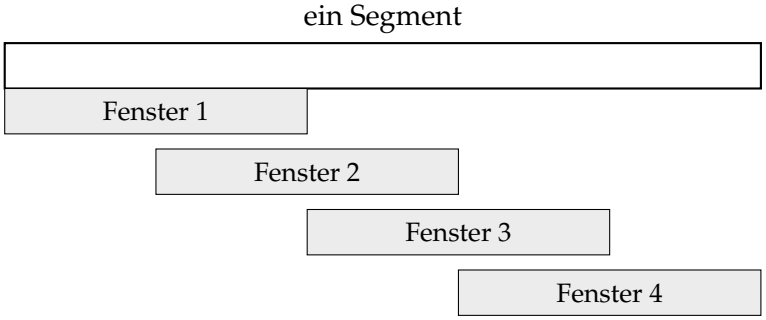


Abbildung 4.2: Zerlegung eines Segments in überlappende Analysefenster (50 % Überlappung) für die Welch-Methode.

4.1.3 Zeitstatistische Merkmale

Neben den spektralen Merkmalen gibt es auch die zeitstatistischen Merkmale. Sie werden für jeden Abschnitt berechnet [1]. Hierfür ist keine FFT notwendig, da die Merkmale direkt aus dem Zeitsignal extrahiert werden [1]. Ein zentrales Merkmal ist die Spannweite

(Peak-to-Peak-Amplitude), welche die Differenz zwischen dem größten und dem kleinsten Amplitudenwert beschreibt [13]. Der Vorteil dieser Herangehensweise ist, dass sie im Gegensatz zur Betrachtung eines absoluten Maximalwertes die Unabhängigkeit von einem potenziellen Gleichanteil des Signals gewährleistet. Aus diesem Grund stellt die Peak-to-Peak-Amplitude ein robustes Kriterium dar, um extreme Signalausschläge und Drifts zu identifizieren, wie sie bei Artefakten auftreten können [13].

Des Weiteren wird zur Beurteilung der Signalqualität der Anteil der ungültigen Messwerte betrachtet. Dies ist der Anteil der Datenpunkte, die vom Erfassungsgerät selbst als fehlerhaft markiert wurden. Man kann durch diesen wichtigen Indikator erkennen, ob Messfehler vorliegen, wie beispielsweise ein unzureichender Kontakt an der Kopfhaut [1].

Im Allgemeinen bilden statistische Merkmale ergänzend zu den spektralen Merkmalen eine gute Grundlage, um die Datenqualität zu beurteilen und Artefakte zu erkennen.

4.1.4 Aufbereitung für das Sprachmodell

Im letzten Schritt der Merkmalsextraktion werden die berechneten Merkmale in eine textuelle Form gebracht, die vom Sprachmodell verarbeitet werden kann. Dabei enthält der Text folgende Informationen:

- aggregierte Kennzahlen
- relative Bandleistung der fünf Frequenzbänder (pro Abschnitt)
- Spannweite (Peak-to-Peak-Amplitude)
- Anteil der ungültigen Messwerte

Dabei werden bewusst keine Bewertungen oder Interpretationen in den Text aufgenommen. Alle Informationen sind objektive Messwerte, deren Bedeutung allein dem Sprachmodell überlassen wird. Dadurch wird gewährleistet, dass die eigentliche Analysearbeit vom Modell erledigt wird.

4.2 Modellauswahl

Wie schon in Abschnitt 3.1.1 beschrieben, stehen systemintegrierte Modelle auf dem verwendeten Testgerät nicht zur Verfügung. Daher konzentriert sich diese Arbeit auf frei deploybare Modelle, wie sie in Abschnitt 3.2 beschrieben sind.

Um eine geeignete Evaluierung zu ermöglichen, wird der Einfluss der Modellgröße unter konstanten Ausführungsbedingungen untersucht. Dazu werden zwei Modelle der Gemma-Familie vergleichend betrachtet: zum einen die kompakte Variante Gemma 3 1B [7, 9], zum anderen die parameterstärkere Gemma 2 2B als größeres Referenzmodell [6, 9]. Da beide Modelle nativ über die MediaPipe-Runtime (LLM Inference API) laufen [9], lässt sich der Effekt der Parameteranzahl auf die Systemperformanz methodisch kontrolliert und isoliert untersuchen.

Beide Modelle werden in 8-Bit-Quantisierung und mit identischer Kontextgröße (1280 Token) ausgeführt, sodass sich der Vergleich allein auf die Modellgröße bezieht.

4.3 Anwendungsszenarien

4.4 Messprotokoll

5 Implementierung

5.1 Architektur

Die Implementierung der Anwendung besteht aus den folgenden Komponenten, die in Abbildung 5.1 dargestellt sind:

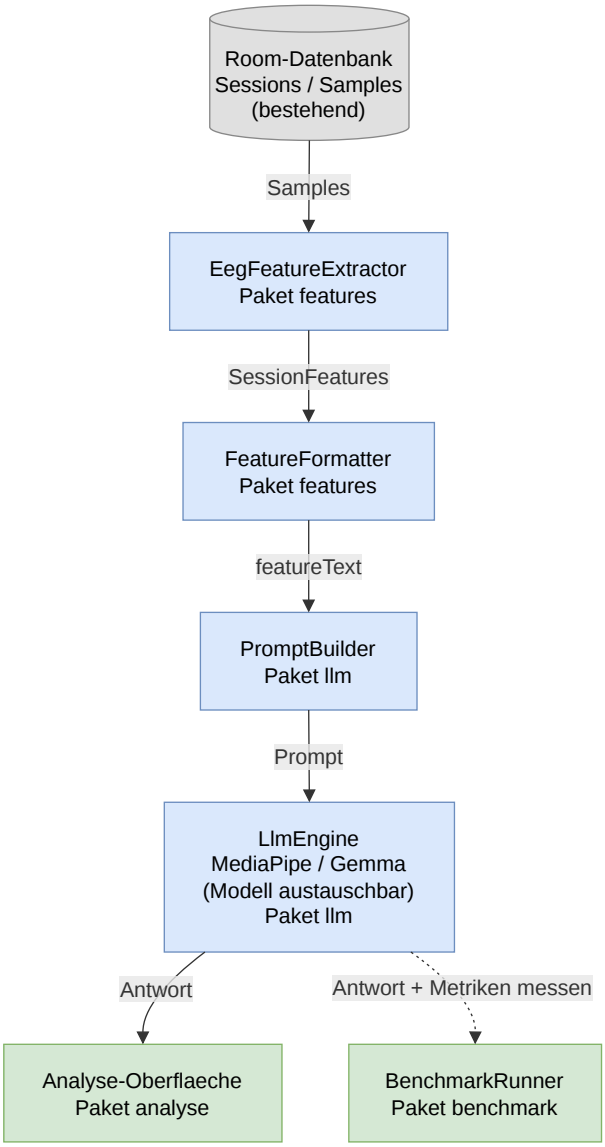


Abbildung 5.1: Architektur und Datenfluss der Anwendung [14]

- **Datenbank:** Der Weg startet über die Datenbank, aus der die Samples abgerufen werden.

- **EegFeatureExtractor:** Hier werden die Samples verarbeitet, indem Merkmale berechnet werden.
- **FeatureFormatter:** Wandelt die Merkmale in ein für das Sprachmodell verständliches Format um.
- **PromptBuilder:** Baut die Anweisung für den Prompt (je Szenario).
- **LlmEngine:** Führt das Modell (Gemma via MediaPipe) aus und gibt die Antwort zurück.
- **Analyse-Oberfläche:** Zeigt letztlich die Antwort an.
- **Benchmark:** Misst die Verarbeitung (Laufzeit, Speicherverbrauch etc.).

Die Architektur ist in drei Kernbereiche unterteilt: zum einen das Paket `features`, das die Merkmalsextraktion übernimmt; zum anderen das Paket `llm`, das je nach Szenario den Aufbau des Prompts übernimmt und das Sprachmodell ausführt; und schließlich das Paket `analyse`, das die Benutzeroberfläche bereitstellt. Durch die Trennung dieser Pakete bleibt der Code übersichtlich, wartbar und erweiterbar.

Ein zentrales Merkmal der Architektur ist die Kapselung des Sprachmodells in der Klasse `LlmEngine`. Die Klasse stellt eine einfache Schnittstelle bereit, die für das Laden, Anfragen und Freigeben verantwortlich ist. Dadurch wird die konkrete Modell- und Laufzeitumgebung vor dem Rest der Anwendung verborgen. Dies ermöglicht das Austauschen der Modelle, ohne dass die anderen Komponenten angepasst werden müssen. In dieser Arbeit werden die Modelle Gemma 3 1B und Gemma 2 2B allein durch den Wechsel des Modellpfads auf dem mobilen Gerät geladen. Aus diesem Grund ließe sich an derselben Schnittstelle auch ein anderes Modell anbinden. Möglich wäre es auch, eine andere Laufzeitumgebung oder ein systemintegriertes Modell anzubinden. Diese Entkopplung ermöglicht es, die Anwendung flexibel und ohne großen Aufwand zu erweitern und zu warten.
Die in Abbildung 5.1 grün und blau markierten Komponenten stellen den eigenen Beitrag dar.

5.2 Implementierung der Merkmalsextraktion

Die Merkmalsextraktion ist im Paket `features` implementiert; zuständig dafür ist die Klasse `EegFeatureExtractor`. Eine Session wird als Erstes aus der Datenbank geladen. Dabei wird geprüft, ob die gesamte Session die Mindestlänge erfüllt und nicht zu stark durch ungültige Messwerte gestört ist. Ist dies nicht der Fall, wird die Analyse abgebrochen und eine entsprechende Meldung angezeigt. Andernfalls wird sie in gleich große Zeitabschnitte aufgeteilt. Dabei werden zwei verschiedene Arten von Merkmalen berechnet: zum einen spektrale Merkmale, also die Leistung in den Frequenzbändern Delta (δ), Theta (θ), Alpha (α), Beta (β) und Gamma (γ); zum anderen statistische Merkmale wie Mittelwert, Standardabweichung und der Anteil ungültiger Werte. Die Klasse `SessionFeatures` dient als Objekt, in dem alle Merkmale gesammelt werden. Schlussendlich wird alles vom `FeatureFormatter` in einen lesbaren Text umgewandelt, den das Sprachmodell verarbeiten kann.

5.3 Integration des Sprachmodells

5.4 Benutzeroberfläche

6 Evaluation

6.1 Versuchsaufbau

6.2 Latenz

6.3 Ressourcenverbrauch

6.4 Stabilität

6.5 Ausgabequalität

7 Diskussion

7.1 Einordnung der Ergebnisse

7.2 Einschränkungen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

8.1 Fazit

8.2 Ausblick

Literatur

[1] Nisreen Said Amer und Samir Brahim Belhaouari. „EEG Signal Processing for Medical Diagnosis, Healthcare, and Monitoring: A Comprehensive Review“. In: *IEEE Access* 11 (2023), S. 143116–143142. DOI: 10.1109/ACCESS.2023.3341419.

[2] Sarah Blum, Stefan Debener, Reiner Emkes, Nils Volkening, Sebastian Fudickar und Martin G. Bleichner. „EEG Recording and Online Signal Processing on Android: A Multiapp Framework for Brain-Computer Interfaces on Smartphone“. In: *BioMed Research International* 2017 (2017), S. 3072870. DOI: 10.1155/2017/3072870.

[3] g.tec medical engineering. *Unicorn Bluetooth Protocol*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black/blob/master/Unicorn%20Bluetooth%20Protocol/UnicornBluetoothProtocol.pdf (besucht am 02. 07. 2026).

[4] g.tec medical engineering. *Unicorn Hybrid Black – User Manual*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black-User-Manual/blob/main/UnicornHybridBlack.md (besucht am 02. 07. 2026).

[5] g.tec medical engineering. *Unicorn Hybrid Black BCI Platform*. URL: https://www.gtec.at/product/unicorn-hybrid-black-bci-platform/ (besucht am 02. 07. 2026).

[6] Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size*. Techn. Ber. Google DeepMind, 2024. URL: https://arxiv.org/abs/2408.00118.

[7] Gemma Team. *Gemma 3 Technical Report*. Techn. Ber. Google DeepMind, 2025. URL: https://arxiv.org/abs/2503.19786.

[8] Google. *Gemini Nano*. 2024. URL: https://developer.android.com/ai/gemini-nano (besucht am 24. 06. 2026).

[9] Google. *LLM Inference Guide – MediaPipe*. 2024. URL: https://developers.google.com/edge/mediapipe/solutions/genai/llm_inference (besucht am 24. 06. 2026).

[10] Google. *Overview of the ML Kit GenAI APIs*. 2025. URL: https://developers.google.com/ml-kit/genai (besucht am 09. 07. 2026).

[11] Qingzhe Guo, Tu Ouyang und An Wang. „Large Language Models on Mobile Devices: A Measurement Study of Single- and Multi-Instance Execution“. In: *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models (EdgeFM '25)*. Hong Kong, China: ACM, 2025. DOI: 10.1145/3737902.3768359.

[12] IBM. *Was ist ein Kontextfenster?* 2024. URL: https://www.ibm.com/de-de/think/topics/context-window (besucht am 12. 07. 2026).

[13] Mainak Jas, Denis A. Engemann, Yousra Bekhti, Federico Raimondo und Alexandre Gramfort. „Autoreject: Automated artifact rejection for MEG and EEG data“. In: *NeuroImage* 159 (2017), S. 417–429. DOI: 10.1016/j.neuroimage.2017.06.030.

[14] JGraph Ltd. *diagrams.net (draw.io)*. URL: https://www.drawio.com/ (besucht am 04. 07. 2026).

[15] Dah-Jing Jwo, Wei-Yeh Chang und I-Hua Wu. „Windowing Techniques, the Welch Method for Improvement of Power Spectrum Estimation“. In: *Computers, Materials & Continua* 67.3 (2021). DOI: 10.32604/cmc.2021.014752.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar u. a. „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[17] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu und Jiming Chen. „A Review on Edge Large Language Models: Design, Execution, and Applications“. In: *ACM Computing Surveys* 57.8 (2025). DOI: 10.1145/3719664.

Abbildungsverzeichnis

4.1 Aufteilung einer Session in vier gleich lange Zeitabschnitte. 13
4.2 Zerlegung eines Segments in überlappende Analysefenster (50 % Überlappung) für die Welch-Methode. 14

5.1 Architektur und Datenfluss der Anwendung [14] 17

Tabellenverzeichnis

2.1 Die fünf EEG-Frequenzbänder und ihre assoziierten Zustände [1] 4

3.1 Aufbau eines Samples des Unicorn Hybrid Black [3] 10

Listings

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der LaTeX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt